

Md. Al Mamun Mim

Senior Full-Stack Engineer — Node.js · NestJS · Next.js · MongoDB · PostgreSQL

Dhaka, Bangladesh (UTC+6) · Available across EU hours and US East mornings

md.almamun.mim.dev@gmail.com · +880 1770540432

[almamun.codes](#) · github.com/MamunCrafts · linkedin.com/in/mamuncrafts

What I'm hired for

I fix slow APIs and fragile data pipelines.

Four years building and running a consumer social-commerce product with 600K registered accounts — which means I've spent that time on the unglamorous end of engineering: query plans that scan entire collections, third-party APIs that change without warning, and infrastructure whose cost curve bends the wrong way.

I read the codebase before I quote. I tell you when something is a tuning problem and when it's an architecture problem, because those need different budgets.

Available: remote contract, hourly or project-based. 1 month notice from current role.

Where I help most

Performance rescue. Slow endpoints, N+1 queries, missing indexes, bloated bundles. I diagnose with query plans and production monitoring rather than guesses, and I show you the before and after.

API and backend architecture. NestJS/Node with REST or GraphQL. Schema design that survives data growth instead of falling over at ten times the volume.

Data pipelines and integrations. ETL against third-party SaaS APIs that will throttle you, change their schema, and send you malformed records. Idempotency, retry with backoff, failure recovery.

Cost-driven re-architecture. When the bill is the bug — recognising a pricing model you cannot optimise your way out of, and making the build-vs-buy call.

Selected work

Moved chat off Firestore onto in-house WebSockets

Chat ran on Firebase/Firestore, which bills per operation. A chat product generates operations relentlessly — every message, every presence update, every listener — so the cost scaled with engagement. The product got more expensive precisely as it got more successful.

That is a structural problem, not a tuning problem: you cannot optimise your way out of a pricing model. I rebuilt chat in-house on WebSockets, using a **Redis pub/sub adapter** so messages reach clients connected to any server instance. Removed the per-operation cost, and brought message ordering and delivery under our own control.

Node.js · WebSockets · Redis pub/sub

Took API endpoints from over five seconds down

Users were reporting slowness. I ran MongoDB **explain()** on the suspect queries and found **full collection scans** — documents examined far exceeding documents returned. Root cause was a query filtering and sorting on an unindexed field: fine when the collection was small, linear-cost disaster once it grew.

Added covering indexes (COLLSCAN → IXSCAN), then redesigned the schema so the access pattern no longer needed the expensive query shape. Production monitoring showed roughly **45% lower latency and 2× throughput** on the core API. Added slow-query alerting afterwards, so the next one surfaces from monitoring instead of from users.

MongoDB · NestJS · GraphQL

Built an ETL / Reverse ETL platform against 7+ third-party APIs

ConversaAI — internal data integration platform moving data between Shopify, WooCommerce, Zoho, Brevo, Mailchimp, QuickBooks, and Wix Commerce.

Every one of those providers will eventually change its schema, rate-limit you mid-sync, or send you malformed records — and punish you for retrying without idempotency keys. I built the pipelines and the reliability layer around them: failure handling, recovery, and safe retries.

Next.js · NestJS · PostgreSQL · Python · Airbyte · AWS EC2

Video transcoding at ~1,000 uploads/day

Ran the transcoding pipeline for a video-first social product. Dealt with queue backpressure, jobs that failed silently, storage growth from multiple renditions, and format-specific failures. The lesson that stuck: a pipeline failing is survivable, a pipeline failing *without telling anyone* is not — users lose content and nobody knows.

Node.js · AWS S3 · queue-based workers

GraphQL API and data layer, 600K registered users

Designed and shipped the GraphQL schema in NestJS serving feed, profiles, comments, and real-time notifications for an Android social commerce app: **1M+ downloads, 600K+ registered accounts, ~30K monthly actives**. Added a Redis caching layer on read-heavy endpoints and cut API payload sizes to reduce mobile data use on slow networks.

NestJS · GraphQL · MongoDB · Redis · TypeScript

Front-end performance

- **Admin console** (Next.js, TypeScript, MUI, Redux) for the operations team — code-splitting and dependency pruning cut initial bundle size ~40% and load time ~25%
- **Marketing sites** — improved Lighthouse mobile usability from **70 to 92**, and cut first-paint time by lazy-loading assets and removing third-party scripts

Stack

Backend: Node.js · NestJS · GraphQL · REST · WebSockets · Python (ETL) **Frontend:** TypeScript · React · Next.js · Redux · MUI · Tailwind **Data:** MongoDB (schema design, indexing, query plans) · PostgreSQL · Redis · Firebase **Infra:** AWS (EC2, S3, CloudWatch) · Docker · Nginx · GitHub Actions CI/CD **Testing & tooling:** Jest · Git · Jira · PostHog · Google Analytics · Airbyte

Certification: MongoDB Certified Associate Developer

Experience

Talent Pro (Simura Group) — Dhaka, Bangladesh · Aug 2022 – present

Joined as an intern, promoted three times to Senior Software Developer (Jan 2026). Team of 20 engineers; **5 engineers report to me** for day-to-day work — I assign tasks, review their PRs, and own technical direction on new services.

Primary product: **Fanfare**, an Android social commerce app combining a social feed, video contests, a gamified points system, and in-app purchasing, selling own-inventory products alongside brand advertising and collaborations.

Side project

RetryDock — self-hosted, multi-tenant webhook reliability platform. In active development; full software requirements spec written first (July 2026) because retry semantics and delivery-state transitions needed settling before any code.

Captures and persists every inbound event before processing, delivers to multiple destinations, retries on a backoff schedule with jitter, parks exhausted deliveries in an inspectable dead-letter queue, and supports replay without mutating history.

It exists because I kept hand-rolling the same reliability layer while fighting flaky third-party APIs on ConversaAI.

Background

BSc, Information and Communication Engineering — University of Rajshahi (2017–2022)

Competitive programming team contestant and leader, ICE Dept. — 600+ problems solved on Codeforces, 100+ on LeetCode. IEEE Xtreme participant, 5th place domestic. Tech Genius Award, Talent Pro.

Languages: English (professional working) · Bengali (native) · Hindi (professional working)